GESTIÓN LEAN-ÁGIL & CALIDAD DE SOFTWARE

Resumen Integrado de Unidades 2, 3 y 4 | Escalado, Métricas, DevOps y PPQA

MÓDULO 1: UNIDAD 2 - GESTIÓN LEAN-ÁGIL DE PRODUCTOS Y ESCALADO

1. Framework para Escalar SCRUM: Nexus

Cuando las dimensiones de un sistema de software o la complejidad de una organización exigen la participación de múltiples equipos de desarrollo trabajando simultáneamente sobre la misma base de código, la coordinación informal resulta insuficiente. Surgen así los marcos de escalado ágil.

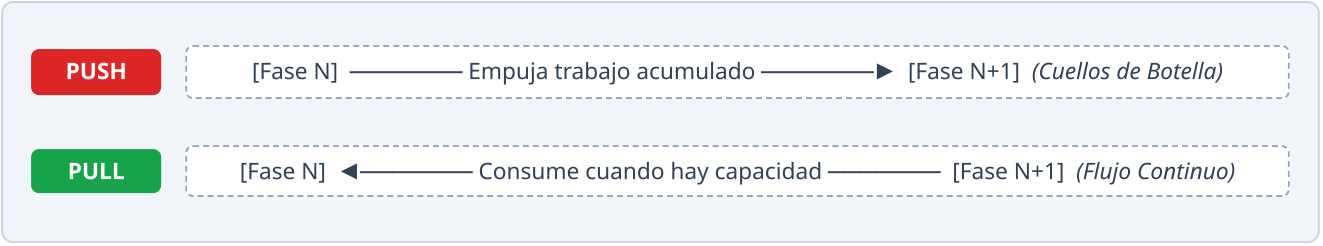
Entre las alternativas principales como **Scrum de Scrums**, **LeSS (Large-Scale Scrum)** y **SAFe (Scaled Agile Framework)**, destaca el framework **Nexus**, desarrollado por Ken Schwaber (co-creador de Scrum).

Nexus está diseñado para coordinar de **3 a 9 equipos Scrum** que trabajan interdependientemente sobre un **único Product Backlog** para entregar al menos un **Incremento de software integrado** al finalizar cada Sprint.

- **El Equipo de Integración Nexus (Nexus Integration Team - NIT):** Actúa como un órgano de liderazgo y facilitación compuesto por el Product Owner, un Scrum Master y miembros representativos de los equipos Scrum. El NIT no ejecuta la construcción directa, sino que asume la responsabilidad total de asegurar que se entregue un incremento integrado utilizable, identificando y resolviendo dependencias cruzadas de arquitectura, código e infraestructura.
- **Eventos Escalados de Nexus:**
  - **Nexus Sprint Planning:** Representantes de todos los equipos inspeccionan el Product Backlog compartido, identifican dependencias entre historias y refinan el trabajo antes de que cada equipo realice su propia planificación de Sprint.
  - **Nexus Daily Scrum:** Representantes de cada equipo se reúnen diariamente para revisar el estado del Incremento Integrado, visibilizar bloqueos cruzados e integrar cambios de código.
  - **Nexus Sprint Review y Retrospective:** Se inspecciona de forma unificada el Incremento integrado completo entregado al cliente, analizando desafíos para adaptar el proceso global.

2. Filosofía Lean y el Método Kanban en el Desarrollo de Software

La **Filosofía Lean**, derivada del Sistema de Producción Toyota, se fundamenta en maximizar el valor entregado al cliente eliminando sistemáticamente el desperdicio (*muda*). En el contexto del software, Lean establece siete principios esenciales: eliminar el desperdicio, amplificar el aprendizaje mediante ciclos iterativos, decidir lo más tarde posible, entregar lo más rápido posible, empoderar al equipo, construir calidad integrada (*build quality in*) y ver el sistema holísticamente.



Origen y Valores de Kanban: Kanban surge a fines de 1940 en el sistema "Just in Time" de Toyota, inspirado en la gestión visual de stock en supermercados mediante tarjetas (*Signal-cards*). Se sostiene sobre tres valores fundamentales: **Transparencia, Equilibrio y Colaboración**.

Principios Fundamentales de Kanban (David J. Anderson):

- **Principios de Gestión del Cambio:** Empezar con lo que estés haciendo AHORA (sin alterar roles ni procesos de un día para otro); acordar buscar la mejora a través del cambio evolutivo; y fomentar el liderazgo en cada nivel de la organización.
- **Principios de Despliegue del Servicio:** Entender y focalizarse en las necesidades y expectativas de los clientes; gestionar el trabajo dejando que la gente se auto-organice alrededor de las tareas; y evolucionar las políticas para mejorar los resultados.

Las 6 Prácticas Concretas de Kanban:

1. **Visualizar el Trabajo:** Representar el flujo de desarrollo en un tablero Kanban dividiendo el proceso en columnas explícitas (ej: Cola de Producto, Análisis, Desarrollo, Testing, Listo para Despliegue) y utilizando tarjetas para cada unidad de trabajo.
2. **Limitar el Trabajo en Progreso (WIP Limits):** Restringir cuantitativamente la cantidad máxima de tarjetas permitidas en cada columna. Al limitar el WIP se fuerzan las restricciones de capacidad, sacando a la luz los cuellos de botella ocultos y reduciendo directamente el *Lead Time*.
3. **Gestionar el Flujo (*Flow*):** Monitorear y optimizar la velocidad y suave transitabilidad de los elementos a través del sistema desde la idea hasta el cliente, buscando un flujo continuo centrado en el valor.
4. **Explicitar Políticas y Acuerdos:** Definir reglas claras de funcionamiento visibles para todos, tales como los criterios de tracción (*Definition of Ready* y *Definition of Done*), horarios de reunión y **Clases de Servicio** (Estándar, Fecha Fija, Expreso/Urgente y Material Inmaterial/Deuda Técnica).
5. **Implementar Bucles de Retroalimentación:** Establecer cadencias operativas regulares para la sincronización y la revisión del estado (revisiones de operaciones, retrospectivas).
6. **Mejorar Colaborativamente y Evolucionar Científicamente:** Utilizar modelos y datos empíricos para experimentar mejoras en un entorno de experimentos seguros ante fallos (*safe-to-fail experiments*).

Conceptos Clave de Flujo:

- **Lead Time:** El tiempo transcurrido desde que un cliente solicita una funcionalidad hasta que se encuentra entregada en producción.
- **Cycle Time:** El tiempo activo en que el equipo trabaja activamente sobre la funcionalidad.
- **Scrumban:** Modelo híbrido que combina la estructura de roles y eventos de Scrum con el flujo continuo y los límites de WIP de Kanban.

3. Métricas de Software en los 3 Enfoques de Gestión

Las métricas en el desarrollo de software adquieren distintas funciones y filosofías según el modelo de proceso sobre el que se apliquen.

> ***Regla de Oro Ágil sobre Métricas:*** *"La medición es una salida, no una actividad. Medir lo que sea necesario y nada más, recordando que el software funcionando es la principal medida de progreso."*

Característica	Enfoque Tradicional	Enfoque Lean	Enfoque Ágil
Premisa	Procesos definidos, predictivos y rígidos.	Optimización de flujo, eliminación de desperdicio.	Aprendizaje empírico, entregas iterativas.
Métricas Clave	- Tamaño: LOC, PF - Desvío Presupuesto/Time - Densidad de Defectos	- Lead Time & Cycle Time - Throughput (Rendimiento) - Flujo Acumulado	- Velocidad (Velocity) - Capacidad (Capacity) - RTF (Tested Features)
Propósito	Control administrativo, auditoría y contrato.	Identificar cuellos de botella y tiempos muertos.	Planificación táctica, autocorregir estimaciones.

Los 3 Dominios de las Métricas de Software:

1. **Nivel Desarrollador (Ámbito Privado / Producto):** Métricas para autoevaluación del ingeniero (esfuerzo, cobertura de pruebas unitarias, defectos en *peer reviews*). *Jamás deben utilizarse para juzgar o castigar el rendimiento individual.*
2. **Nivel Equipo de Desarrollo (Ámbito del Proyecto):** Métricas tácticas para monitorear la salud de la iteración (Velocidad, Capacidad, estabilidad de requerimientos).
3. **Nivel Organización (Ámbito de Proceso / Empresa):** Métricas estratégicas para evaluar el ROI corporativo (precisión de estimaciones, tasa de defectos en *releases*).

La Métrica "Concept to Cash" en Lean: Métrica propuesta por Mary y Tom Poppendieck que mide el tiempo total desde que el cliente solicita una funcionalidad hasta que el software entregado genera valor comercial real.

MÓDULO 2: UNIDAD 3 - GESTIÓN DE SOFTWARE COMO PRODUCTO Y PRÁCTICAS CONTINUAS

1. Gestión de Software como Producto (Product Thinking)

A diferencia del enfoque de proyectos tradicional (donde el objetivo es la eficiencia en cumplir un presupuesto y calendario acotados), la **Gestión de Productos** busca la **eficacia estratégica: construir el producto correcto que genere impacto real en el cliente**. Los proyectos terminan; los productos son activos vivos que evolucionan a lo largo de todo su ciclo de vida.

El cambio fundamental reside en pasar de medir únicamente **Outputs** (salida técnica: código entregado o historias completadas) a medir **Outcomes** (impacto de negocio: cambios observables en el comportamiento del usuario y retorno de inversión).

2. Prácticas Continuas (Extreme Programming y DevOps)

Para sostener la entrega continua de valor en un activo vivo de software, se requiere un conjunto disciplinado de prácticas técnicas continuas:

- **Integración Continua (CI - Continuous Integration):** Práctica en la que todos los desarrolladores integran su trabajo en el repositorio central compartido con alta frecuencia (al menos una vez al día). Cada *commit* activa un servidor automatizado que descarga el código, ejecuta la construcción (*build*) y corre de forma autónoma la batería de pruebas. Esto erradica el fenómeno de *"funciona en mi máquina"*.
- **Entrega Continua (Continuous Delivery):** Extensión de CI donde la aplicación se mantiene en todo momento en un estado técnicamente desplegable hacia entornos de *staging* o pre-producción mediante pipelines automatizados.
- **Despliegue Continuo (Continuous Deployment):** Automatización avanzada en la cual cada cambio que supera exitosamente todas las fases del pipeline de integración se despliega directamente a producción.
- **Prácticas Complementarias de Calidad Técnica:**
  - **Refactorización Constante:** Limpieza continua del código fuente para mejorar su diseño interno sin alterar su comportamiento externo.
  - **Diseño Incremental:** Evitar el diseño rígido inicial, permitiendo que la arquitectura evolucione con las funcionalidades.
  - **Propiedad Colectiva del Código:** Cualquier desarrollador puede modificar o mejorar cualquier parte del código fuente.

MÓDULO 3: UNIDAD 4 - ASEGURAMIENTO DE CALIDAD DE PROCESO Y DE PRODUCTO (PPQA)

1. Conceptos Generales sobre Calidad

De acuerdo con los estándares ISO y IEEE, la **Calidad del Software** es el grado en que un sistema, componente o proceso cumple con los requisitos especificados y con las necesidades y expectativas explícitas e implícitas del cliente o usuario final.

Es imprescindible distinguir la jerarquía entre tres términos fundamentales:



Los 4 Tipos de Mantenimiento de Software:

La actitud correcta hacia la calidad en la ingeniería de software es la prevención: es mucho mejor evitar los problemas desde el inicio. Cuando la prevención falla, se recurre a acciones de mantenimiento:

- **Mantenimiento Correctivo:** Modificación reactiva tras la entrega para corregir fallas descubiertas.
- **Mantenimiento Adaptativo:** Modificación post-entrega para mantener el producto utilizable dentro de un entorno operativo que ha cambiado.
- **Mantenimiento Perfectivo:** Modificación para mejorar la mantenibilidad o el rendimiento (*performance*) del sistema.
- **Mantenimiento Preventivo:** Modificación proactiva para detectar y corregir defectos latentes en el código antes de que se conviertan en fallas reales.

Calidad de Proceso vs. Calidad de Producto:

- **Calidad de Proceso (Enfoque Preventivo):** Orientada a garantizar que las actividades, métodos y técnicas utilizados para construir el software sean consistentes y disciplinados. Un proceso maduro minimiza la introducción de errores.
- **Calidad de Producto (Enfoque Detectivo y Evaluativo):** Orientada a examinar las características técnicas y funcionales del entregable final para verificar que cumpla con los estándares esperados.

> **Matriz de Relaciones de Factores de Calidad (Trade-offs):**
> La calidad del software no es unidimensional; abarca múltiples atributos (Correctitud, Usabilidad, Eficiencia, Fiabilidad, Integridad, Adaptabilidad, Precisión y Robustez) que frecuentemente entran en conflicto entre sí. El aseguramiento de calidad exige gestionar estos sacrificios. Por ejemplo, priorizar la *Eficiencia* suele perjudicar la *Adaptabilidad* y la *Robustez*; mientras que priorizar la *Correctitud* estricta mejora la *Fiabilidad*, pero suele restringir la *Robustez* frente a situaciones atípicas.

2. Importancia de Trabajar con Calidad

Ventajas de la Calidad:

- **Reducción Exponencial de Costos de Retrabajo:** Detectar y corregir un defecto en las fases iniciales cuesta una fracción insignificante en comparación con corregirlo durante las pruebas de sistema o en producción.
- **Mayor Satisfacción y Retención de Usuarios:** Un producto fiable y seguro construye reputación.
- **Mantenibilidad y Sostenibilidad:** Código limpio reduce la entropía técnica del sistema.

Inversiones Requeridas (Desventajas): Costo inicial para herramientas de prueba/capacitación, y posible fricción burocrática en enfoques tradicionales si los procesos no se automatizan adecuadamente.

3. Actividades del Aseguramiento de la Calidad (PPQA)

La disciplina de **PPQA (Process and Product Quality Assurance)** actúa como actividad sombrilla a lo largo del ciclo de vida. Sus actividades principales abarcan:

1. **Establecimiento del Plan de Calidad:** Definir estándares, métricas y procedimientos.
2. **Evaluación Objetiva de Procesos:** Revisar adecuación a procesos definidos.
3. **Evaluación Objetiva de Productos de Trabajo:** Inspeccionar artefactos (código, documentos).
4. **Seguimiento y Reporte de Desviaciones:** Comunicar hallazgos a la gerencia.

4. Principales Modelos de Calidad Existentes

Modelo / Estándar	Descripción y Enfoque	Niveles o Estructura
CMMI (SEI)	Evalúa la madurez organizacional y capacidad de procesos. Enfoque descriptivo y cuantitativo.	5 Niveles: Initial, Managed, Defined, Quantitatively Managed, Optimizing.
SPICE (ISO/IEC 15504)	Estándar internacional de evaluación de capacidad de procesos de software.	Dimensión de Proceso (ISO 12207) y Dimensión de Capacidad (Niveles 0 a 5).
ISO 9001 / TickIT	Gestión de calidad genérica adaptada al software mediante auditorías de certificación.	Basado en el ciclo PHVA (Planificar, Hacer, Verificar, Actuar).
ISO/IEC 25010 (SQuaRE)	Modelo de Calidad de Producto de Software (evalúa el software entregado directamente).	8 Características: Funcionalidad, Usabilidad, Eficiencia, Fiabilidad, etc.

CMMI vs. Métodos Ágiles:

- **CMMI:** Enfoque descriptivo, cuantitativo, universal y estratégico. Centrado en procesos internos, reglas rígidas y gestión proactiva de riesgos mediante documentación formal.
- **Métodos Ágiles:** Enfoque prescriptivo, cualitativo/táctico, situacional y orientado directamente al producto en funcionamiento. Centrado en la innovación externa, la velocidad y la adaptación al cambio.

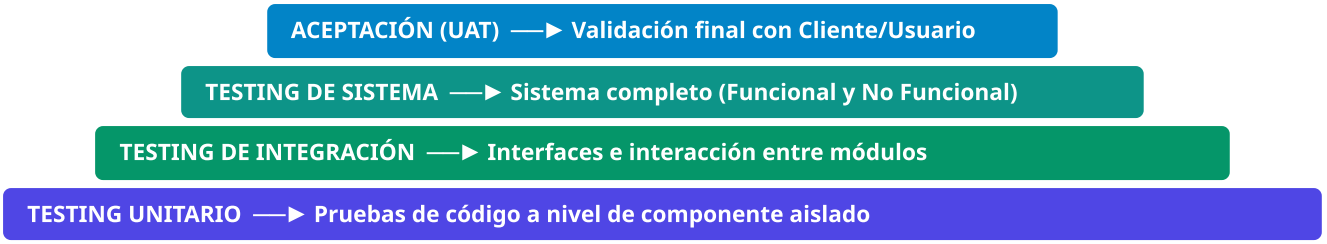
5. Auditorías de Software: Proceso y Tipos

Una **Auditoría de Software** es un examen formal, independiente y sistemático realizado para determinar si las actividades de desarrollo y sus resultados cumplen con las especificaciones y estándares establecidos.

- **Tipos de Auditorías:** Proyecto (gestión operativa), Configuración Funcional FCA (comportamiento lógico), Configuración Física PCA (archivos/entregables físicos), y PPQA (auditoría al propio grupo de calidad).
- **Etapas del Proceso:** Planificación ➔ Preparación ➔ Ejecución ➔ Reporte de Hallazgos ➔ Seguimiento de Acciones Correctivas.

6. Calidad de Producto: Planificación y Niveles

La verificación y validación del software ejecutable se estructuran a través de un proceso de Testing formal.



Estrategias y Técnicas de Pruebas:

- **Pruebas de Caja Blanca (Estructurales):** Basadas en el conocimiento detallado del código fuente. Evalúan la cobertura de rutas lógicas, condiciones y bucles.
- **Pruebas de Caja Negra (Funcionales):** Aplicadas sobre las interfaces externas sin acceder al código interno. Evalúan entradas y salidas frente a los requerimientos.
  - *Técnicas de Caja Negra:* Partición de Equivalencia, Análisis de Valores Límite/Frontera, Tablas de Decisión, Transición de Estados y Arreglos Ortogonales.

7. Revisiones Técnicas del Software (Análisis Estático)

Consisten en la inspección estática de artefactos (documentos de requerimientos, diagramas, código fuente) **sin ejecutar el programa**, con el fin de descubrir errores de forma temprana.

Incluyen **Revisiones Informales (Peer Reviews)**, **Recorridos Técnicos (Walkthroughs)** guiados por el autor, e **Inspecciones Formales (Fagan Inspections)** estructuradas con roles explícitos y listas de chequeo. Se complementan con herramientas automáticas como Linters y analizadores de complejidad ciclomática.

8. Testing en Ambientes Ágiles

En el paradigma ágil, el testing deja de ser una fase tardía realizada al final del proyecto y se convierte en una **actividad continua e intrínseca** que conduce el desarrollo desde el primer día, formando parte innegociable de la *Definition of Done (DoD)*.

1. **Pruebas Automatizadas de Unidad e Integración:** Baterías de pruebas ejecutadas de forma autónoma con cada construcción de software.
2. **Desarrollo Conducido por Pruebas (TDD - Test-Driven Development):** Ciclo iterativo basado en **Red ➔ Green ➔ Refactor** (Escribir test que falle ➔ Escribir código mínimo para que pase ➔ Limpiar y optimizar código). Utiliza patrones como *Fake it*, *Triangulación* y *Explanation tests*.
3. **Pruebas de Regresión Automatizadas a Nivel de Sistema:** Garantizan que los nuevos cambios o adiciones no rompan funcionalidades preexistentes.
4. **Desarrollo Conducido por Pruebas de Aceptación (ATDD):** Colaboración estrecha entre cliente, testers y desarrolladores para definir ejemplos ejecutables antes de programar la historia de usuario.
5. **Pruebas Exploratorias (Exploratory Testing):** Actividad donde el tester diseña y ejecuta pruebas simultáneamente, utilizando la intuición y el aprendizaje dinámico.
6. **Control de Versiones de las Pruebas con el Código:** Los scripts de prueba residen en el mismo repositorio central que el código de producción, evolucionando en estricta sincronía.